

Desafío de Laboratorio: Visualización Geoespacial

Fernan Severich

2026-03-10

Table of contents

0.1	Introducción	1
0.2	Python Estático (Geopandas)	1
0.3	Python Dinámico (Geopandas)	3
0.4	Python Dinámico (Geemap)	4
0.5	R Estático	5
0.6	R Dinámico (mapview)	7
0.7	Parte B: Desafío de Exploración y Análisis	9
0.7.1	Leafmap: Implementar un mapa de pantalla dividida	9
0.7.2	Geemap: Cargar un dataset diferente	9
0.7.3	Análisis	10
0.8	Listado de Archivos y Carpetas (Anexos)	10

0.1 Introducción

En este documento se presentan las respuestas al desafío de visualización geoespacial, integrando herramientas de los ecosistemas Python (OpenGeos) y R (R-Spatial).

i Note

Aviso técnico: Durante el desarrollo de este laboratorio, se presentaron limitaciones técnicas para ejecutar archivos con extensión `.ipynb` utilizando el **Kernel de R** dentro del entorno de VS Code/Jupyter. Por lo tanto, con el objetivo de garantizar la ejecución correcta del código y la visualización de los mapas, se optó por integrar todo el desarrollo (tanto Python como R) directamente en este archivo **Quarto (.qmd)**. Quarto permite la ejecución multilingüe de forma nativa en VSCode, superando las restricciones de kernel encontradas en los notebooks tradicionales.

0.2 Python Estático (Geopandas)

Visualización de los distritos de Nueva York utilizando datos vectoriales locales. En este apartado se utilizó la librería Geopandas junto con el motor de renderizado de Matplotlib para generar una representación cartográfica estática de los límites administrativos de Nueva York. A través del método `.plot()`, se categorizaron los datos vectoriales según el atributo 'BoroName', integrando una leyenda que facilita la diferenciación visual de cada distrito. Esta técnica de visualización es fundamental para la creación

de reportes técnicos y publicaciones académicas, ya que permite comunicar con claridad la distribución espacial de las variables mediante una imagen fija de alta fidelidad y fácil interpretación.

```
# Instala la librería geodatasets.
#!pip3 install geodatasets

# Importa geopandas y geodatasets.
import geopandas
from geodatasets import get_path
import matplotlib.pyplot as plt

# Obtiene la ruta del dataset:
# "nybb" (New York Borough Boundaries).
path_to_data = get_path("nybb")

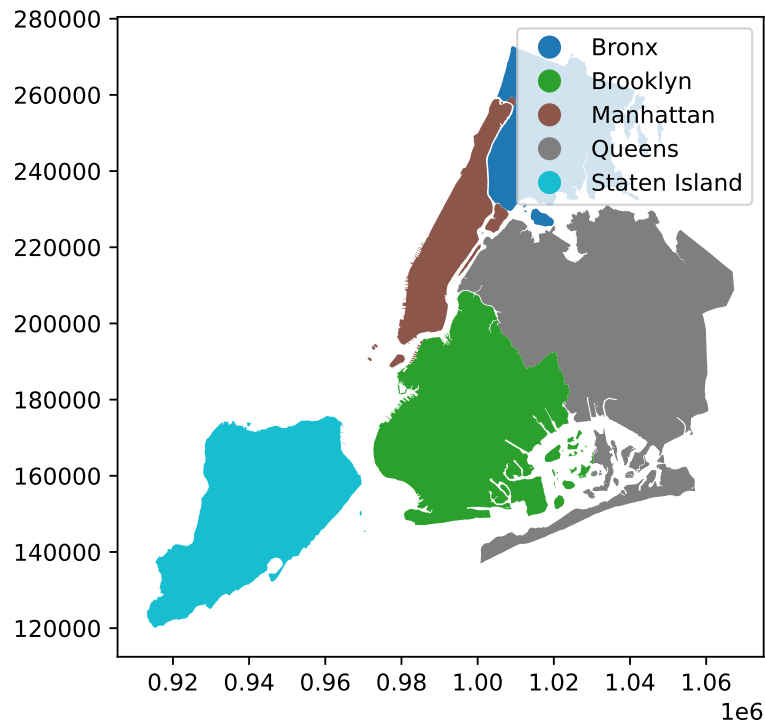
# Lee el archivo y lo convierte
# en un GeoDataFrame
gdf = geopandas.read_file(path_to_data)

# Muestra la tabla.
gdf
```

	BoroCode	...	geometry
0	5	...	MULTIPOLYGON (((970217.022 145643.332, 970227....
1	4	...	MULTIPOLYGON (((1029606.077 156073.814, 102957...
2	3	...	MULTIPOLYGON (((1021176.479 151374.797, 102100...
3	1	...	MULTIPOLYGON (((981219.056 188655.316, 980940....
4	2	...	MULTIPOLYGON (((1012821.806 229228.265, 101278...

[5 rows x 5 columns]

```
# --- Visualización Estática ---
# Genera un mapa fijo usando Matplotlib, coloreando por el nombre del distrito.
# Se incluye una leyenda para identificar cada color con su respectivo 'Borough'.
gdf.plot("BoroName", legend=True, figsize=(8, 5))
plt.show()
```



0.3 Python Dinámico (Geopandas)

Visualización de los distritos de Nueva York Se empleó el método `.explore()` de la librería Geopandas para transformar los datos vectoriales de los distritos de Nueva York en una visualización geoespacial interactiva. A diferencia de los mapas estáticos, esta funcionalidad integra automáticamente el motor de Leaflet, permitiendo superponer la geometría de los datos sobre un mapa base dinámico con capacidades de zoom y desplazamiento. Este enfoque es particularmente valioso en la fase de exploración de datos, ya que facilita la inspección visual de los atributos espaciales (como el 'BoroName') en un contexto geográfico real, optimizando la interpretación de la distribución territorial de forma inmediata y sencilla.

```
# Instala la librería geodatasets.
#!pip3 install geodatasets

# Importa geopandas y geodatasets.
import geopandas
from geodatasets import get_path

# Obtiene la ruta del dataset:
# "nybb" (New York Borough Boundaries).
path_to_data = get_path("nybb")

# Lee el archivo y lo convierte
# en un GeoDataFrame
gdf = geopandas.read_file(path_to_data)
```

```
# Muestra la tabla.
gdf
```

```

  BoroCode  ...      geometry
0         5  ...  MULTIPOLYGON (((970217.022 145643.332, 970227....
1         4  ...  MULTIPOLYGON (((1029606.077 156073.814, 102957...
2         3  ...  MULTIPOLYGON (((1021176.479 151374.797, 102100...
3         1  ...  MULTIPOLYGON (((981219.056 188655.316, 980940....
4         2  ...  MULTIPOLYGON (((1012821.806 229228.265, 101278...

```

```
[5 rows x 5 columns]
```

```
# --- Visualización Dinámica ---
# Crea un mapa interactivo basado en Leaflet.
# El mapa base se carga automáticamente.

m_nyc = gdf.explore("BoroName", legend=False)
m_nyc
```

```
<folium.folium.Map object at 0x725892d0b2c0>
```

0.4 Python Dinámico (Geemap)

Se integró el uso de Google Earth Engine (GEE) mediante la librería geemap para el procesamiento de imágenes satelitales en la nube. A través de este flujo de trabajo, se filtró y procesó una colección de Sentinel-2 enfocada en Bogotá, aplicando una reducción por mediana para generar un mosaico de alta calidad libre de nubes. Esta aproximación demuestra la potencia del ecosistema Python para realizar análisis de sensores remotos a gran escala, permitiendo visualizar datos multiespectrales de forma dinámica sin necesidad de descargar volúmenes masivos de información localmente.

```
import geemap
import ee

# 1. Inicializar conexión
try:
    ee.Initialize()
except Exception as e:
    # En caso de que necesites re-autenticar en Docker
    print("Error de inicialización: Asegúrate de haber corrido ee.Authenticate()")

# 2. Crear el mapa estándar (evitamos folium por el error de la librería)
m_sentinel = geemap.Map(center=[4.63, -74.08], zoom=10)

# 3. Cargar colección Sentinel-2
# Usamos nombres claros para evitar conflictos de variables
coleccion_s2 = (
    ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
```

```

    .filterDate('2024-01-01', '2024-03-31')
    .median()
)

# 4. Parámetros de visualización
# Definimos los valores como enteros simples para evitar errores de JSON
params = {'min': 0, 'max': 3000, 'bands': ['B4', 'B3', 'B2']}

# 5. Añadir capa y mostrar
m_sentinel.addLayer(coleccion_s2, params, 'Sentinel-2 Bogotá')

# Mostrar el objeto directamente
m_sentinel

```

i Note

Se realizaron modificaciones en el archivo .qmd para garantizar la compatibilidad entre lenguajes y la correcta generación del informe final. En primer lugar, se corrigieron los delimitadores de los bloques de código para evitar que el texto explicativo fuera interpretado como sintaxis de Python. Segundo, se ajustaron las rutas de las imágenes locales utilizando barras diagonales (/) para asegurar su lectura en el entorno Linux/Docker. Por último, se resolvió el conflicto de serialización JSON al separar la visualización de tablas de datos de la generación de mapas dinámicos, evitando así errores de traducción entre Python y R a través de la librería reticulate y permitiendo un renderizado fluido de los componentes interactivos.

0.5 R Estático

En el apartado de R-Spatial, se utilizó la combinación de las librerías sf y tmap para generar una visualización temática de Colombia. A partir del dataset global 'World', se realizó un filtrado espacial para representar la variable de población estimada mediante un mapa de coropletas. La composición final integra elementos cartográficos fundamentales, como la escala gráfica y la rosa de los vientos, asegurando una correcta orientación y referencia métrica. Este ejercicio demuestra la capacidad del ecosistema R para producir cartografía estática de alta calidad profesional, manteniendo la precisión técnica y la claridad visual necesarias para el análisis de datos sociodemográficos.

```

library(tmap)
library(sf)

```

Linking to GEOS 3.12.1, GDAL 3.8.4, PROJ 9.4.0; sf_use_s2() is TRUE

```

library(ggplot2)

data("World")
colombia <- World[World$name == "Colombia", ]

tm_shape(colombia) +
  tm_polygons("pop_est", palette = "Blues", title = "Población Est.") +

```

```
tm_compass(position = c("right", "top")) +
tm_scale_bar()
```

-- tmap v3 code detected -----

[v3->v4] `tm\_tm\_polygons()`: migrate the argument(s) related to the scale of the visual variable `fill` namely 'palette' (rename to 'values') to fill.scale = tm_scale(<HERE>).

[v3->v4] `tm\_polygons()`: migrate the argument(s) related to the legend of the visual variable `fill` namely 'title' to 'fill.legend = tm_legend(<HERE>)'

! `tm\_scale\_bar()` is deprecated. Please use `tm\_scalebar()` instead.

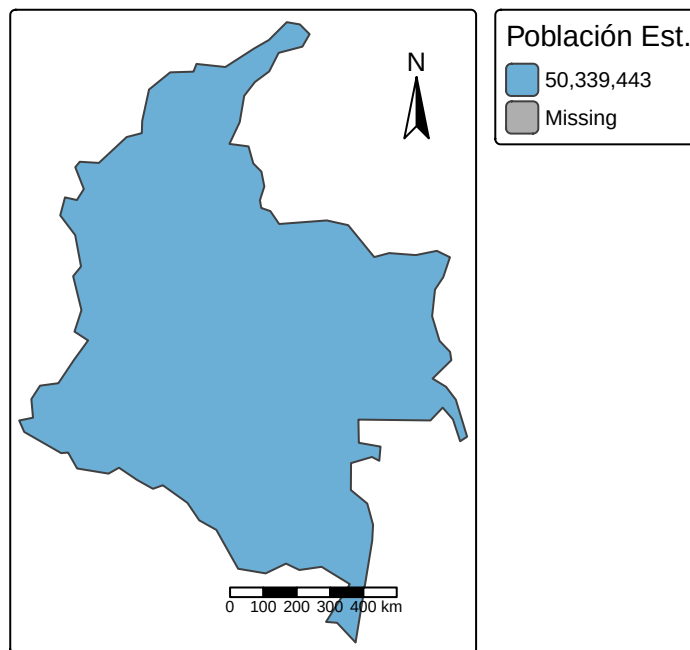
The visual variable `fill` of the layer "polygons" contains a unique value.

Therefore a discrete scale is applied (tm_scale_discrete).

[cols4all] color palettes: use palettes from the R package cols4all. Run

`cols4all::c4a\_gui()` to explore them. The old palette name "Blues" is named "brewer.blues"

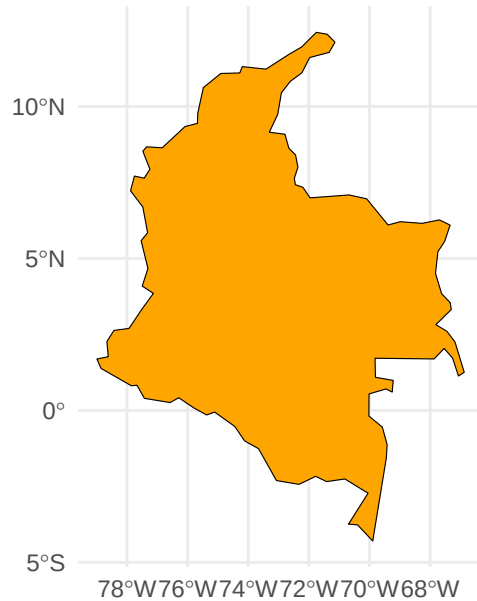
Multiple palettes called "blues" found: "brewer.blues", "matplotlib.blues". The first one, "brewer.blues"



--- Usando ggplot2 (Gramática de gráficos) ---

```
ggplot() +
  geom_sf(data = colombia, fill = "orange", color = "black") +
  theme_minimal() +
  labs(title = "Mapa de Colombia con ggplot2")
```

Mapa de Colombia con ggplot2



Se implementó una visualización adicional utilizando la librería ggplot2 y su función especializada `geom_sf()`, la cual representa el estándar para el manejo de datos espaciales en el entorno tidyverse de R. Esta aproximación destaca por su diseño minimalista y la inclusión automática de una cuadrícula de coordenadas geográficas, demostrando la versatilidad de la gramática de gráficos para generar mapas vectoriales precisos, limpios y altamente personalizables.

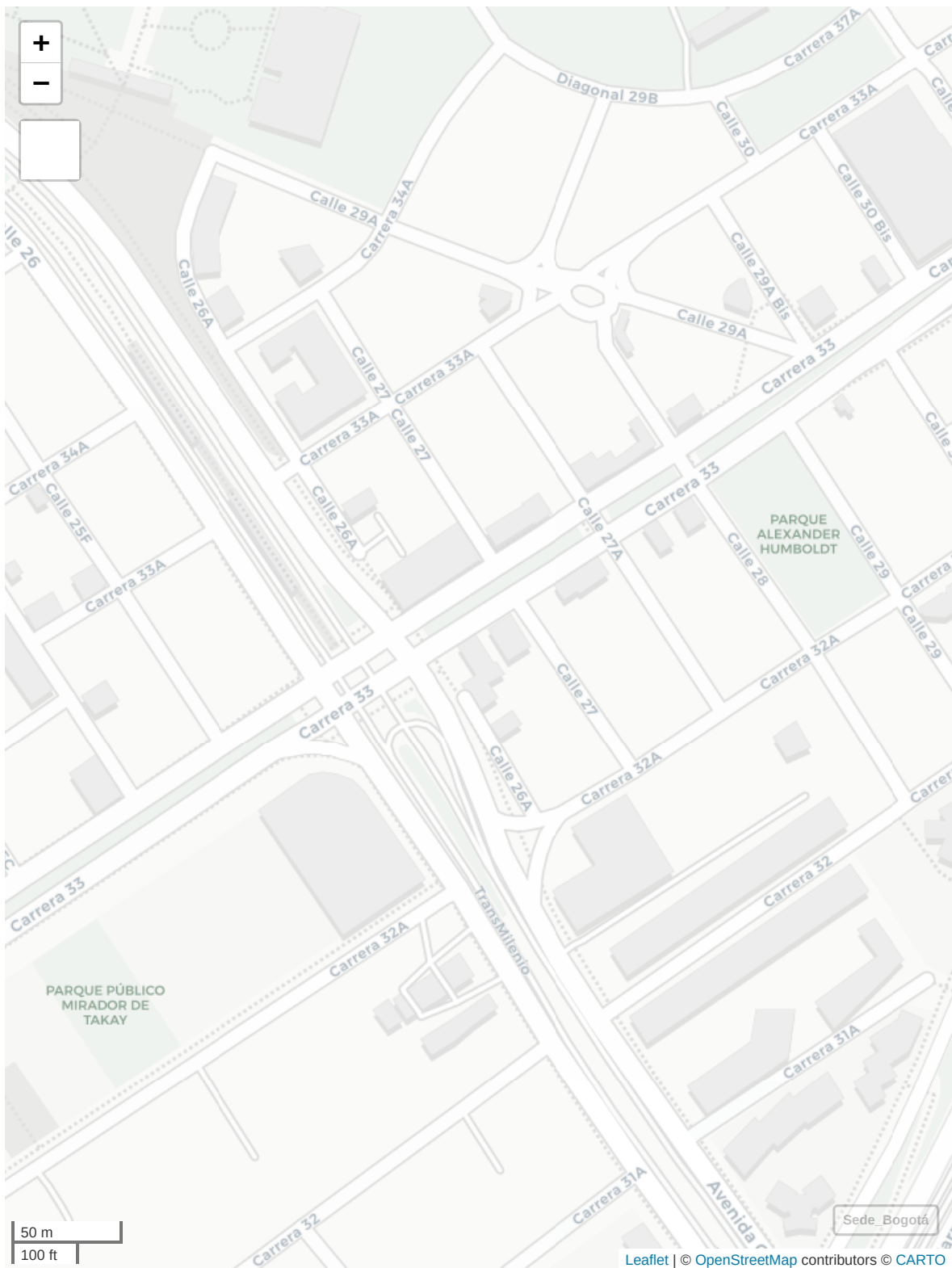
0.6 R Dinámico (mapview)

Para la visualización interactiva en R, se utilizó la librería mapview en conjunto con sf para georreferenciar y representar un punto de interés específico: la sede Bogotá de la Universidad Nacional de Colombia. Esta herramienta destaca por su eficiencia para generar mapas dinámicos de forma inmediata, integrando automáticamente controles de capas y mapas base de OpenStreetMap. Este enfoque permite una exploración rápida y detallada de coordenadas puntuales (EPSG:4326), facilitando la inspección de datos espaciales dentro de un entorno web interactivo y funcional.

```
library(sf)
library(mapview)

# 1. Crear punto de referencia UNAL Bogotá (EPSG:4326)
unal <- st_sfc(st_point(c(-74.08, 4.63)), crs = 4326)

# 2. Visualización interactiva inmediata
# Genera automáticamente controles de capas y mapas base
mapview(unal, col.regions = "red", layer.name = "Sede Bogotá")
```



0.7 Parte B: Desafío de Exploración y Análisis

0.7.1 Leafmap: Implementar un mapa de pantalla dividida

He seleccionado la librería **Leafmap** para demostrar la funcionalidad de **Split-Panel** (Pantalla dividida). Esta herramienta es fundamental para comparar cambios en el territorio (antes/después) o diferentes capas de información.

```
import leafmap.foliummap as leafmap # <--- Cambiamos el motor aquí

# Crear mapa comparativo de Bogotá
m = leafmap.Map(center=[4.60, -74.07], zoom=13)

# En folium, los nombres de capas pueden variar un poco
m.split_map(
    left_layer="TERRAIN",
    right_layer="ROADMAP" # O "SATELLITE"
)

m
```

0.7.2 Geemap: Cargar un dataset diferente

Para este desafío, utilicé la librería geemap para visualizar el Modelo Digital de Elevación SRTM, explorando un dataset de tipo topográfico en lugar de imágenes ópticas. Como funcionalidad diferenciadora a la vista en clase, implementé una paleta de colores personalizada y una barra de escala cromática interactiva (colorbar), lo cual permite la interpretación cuantitativa de las altitudes en la región de Bogotá, transformando una capa visual en un producto cartográfico con valor analítico.

```
import ee
import geemap

# 1. Inicializar Earth Engine
try:
    ee.Initialize()
except Exception as e:
    ee.Authenticate()
    ee.Initialize()

# 2. Cargar el dataset SRTM (Digital Elevation Model)
# Este dataset mide la altura sobre el nivel del mar
srtm = ee.Image('CGIAR/SRTM90_V4')

# 3. Configurar visualización con una paleta tipo relieve (terrain)
# Definimos que el color cambie según la altura (de 0 a 3000 metros)
vis_params = {
    'min': 0,
    'max': 3000,
    'palette': ['blue', 'green', 'yellow', 'orange', 'red', 'white']
}
```

```

}

# 4. Crear el mapa interactivo
m = geemap.Map(center=[4.63, -74.08], zoom=9)

# 5. Añadir la capa de elevación
m.addLayer(srtm, vis_params, 'Elevación SRTM')

# 6. FUNCIONALIDAD DIFERENTE: Añadir una barra de colores (Colorbar)
# Esto ayuda al usuario a entender qué altura representa cada color
m.add_colorbar(vis_params, label="Elevación (m)", orientation="horizontal", layer_name="SRTM")

```



```

# Mostrar el mapa
m

```

0.7.3 Análisis

1. A mi parecer, Leafmap es la más intuitiva para un principiante. A diferencia de Geemap, no requiere lidiar con la autenticación de Google Earth Engine inicialmente y permite cargar archivos locales (como GeoJSON o Shapefiles) con una sola línea de código, lo que reduce la frustración inicial de quien no sabe programar, sin embargo las guías actuales con inteligencia artificial, facilita mucho no quedarse atascado en un punto como este.
2. Para un informe técnico en PDF, elegiría tmap (si se usa R) o matplotlib (geopandas.plot) en Python. El motivo es que los mapas dinámicos son excelentes para explorar, pero estos motores permiten un control sobre la simbología, leyendas estáticas, escala gráfica y flecha de norte, que son requisitos esenciales en cartografía profesional estática.
3. Al revisar OpenGeos, encontré que la funcionalidad de Análisis de series temporales es potente. Permitiría, por ejemplo, ver cómo ha crecido la mancha urbana de Bogotá en los últimos 20 años de forma automatizada, algo vital para mi investigación sobre expansión urbana.
4. En cuanto a la elección de Herramientas en temas de Emergencia Ambiental Elegiría Leafmap o Geemap junto con Streamlit permiten crear un visor web interactivo en menos de 10 líneas de código aprovechando datos en la nube (como inundaciones detectadas por Sentinel-1). Su integración con widgets de control hace que personal no técnico pueda activar/desactivar capas fácilmente sin ver una sola línea de código.

0.8 Listado de Archivos y Carpetas (Anexos)

dinamico_r_files Carpeta de archivos - estatico_r_files Carpeta de archivos - Imag Carpeta de archivos - Taller_clase_2_files Carpeta de archivos - dinamico_r Chrome HTML Document 29 KB
 dinamico_r.qmd Archivo QMD 1 KB estatico_r Chrome HTML Document 21 KB estatico_r.qmd Archivo QMD 1 KB
 py_dinamico Archivo de origen (Python) 2 KB Py_estatico Archivo de origen (Python) 3,152 KB
 Taller_clase_2.aux Archivo AUX 2 KB Taller_clase_2 Chrome HTML Document 12,783 KB
 Taller_clase_2 Documento de texto 69 KB Taller_clase_2 PDF Document 1,008

KB Taller_clase_2.qmd Archivo QMD 13 KB Taller_clase_2 Archivo de origen (Python) 24 KB
Taller_clase_2.toc Archivo TOC 0 KB